

Calibración y Compensación de Sensado de Corriente

En esta parte se indicará el problema encontrado con la medición de corriente así como la solución empleada. Este problema se detectó después del armado de la placa, aunque el circuito de sensado haya sido probado con anterioridad.

1. Diagnóstico del Problema (Hardware)

El diseño original utiliza un **Amplificador Diferencial** para medir la caída de tensión en una resistencia de shunt (1 ohm) ubicada en el **lado de alta tensión (High Side)**.

- **La falla detectada:** El ADC de la Raspberry Pi Pico reportaba un valor casi constante de **~690 mV**, independientemente de si la salida era de 12V o 24V.
- **La causa raíz:** El operacional (un LM358 o similar) operando en modo *High Side* estaba trabajando fuera de su **Rango de Tensión en Modo Común**. Al estar las entradas tan cerca del riel de alimentación, la salida se "saturaba" en un piso de voltaje (offset) muy alto, perdiendo la linealidad teórica de la ganancia.

2. Análisis de Datos (Empírico)

A pesar del error de hardware, los datos medidos mostraron una pequeña ventana de respuesta que pudimos aprovechar:

- A 12V (Esperado 60mA): El sensor entregaba un voltaje equivalente a 65mA.
- A 24V (Esperado 120mA): El sensor entregaba un voltaje equivalente a 125mA.

Esto nos indicó que, aunque el sensor tenía un "piso" de error muy grande, todavía existía una relación lineal aprovechable entre el cambio de corriente y el cambio de voltaje en el ADC.

3. Solución Aplicada: Curva de Transferencia Lineal

En lugar de confiar en la fórmula teórica $I = V_{adc} / (G \times R)$, desarrollamos una ecuación de calibración personalizada basada en las mediciones reales ($y = mx + b$).

Pasos de la solución:

1. Eliminación del Offset: Identificamos que el operacional nunca bajaba de 0.666V. Restamos este valor en el código para establecer el "cero" real.
2. Ajuste de Pendiente (m): Calculamos un factor de escala (1350.0f) para amplificar la pequeña variación de milivoltios detectada y convertirla en una lectura de miliamperios coherente.
3. Filtro Digital: Implementamos un Promedio Móvil de 20 muestras para estabilizar la lectura, ya que al trabajar con rangos tan pequeños, cualquier ruido eléctrico del PWM hacía saltar los valores en el LCD.

4. Resultados Finales

Tras el ajuste, el sistema logró una precisión notable para una placa ya armada:

- Error en 12V: +5mA (8% de error).
- Error en 24V: +5mA (4% de error).

5. Conclusiones y Recomendaciones

El problema fue resuelto exitosamente mediante **procesamiento de señales**. Para futuros diseños (Versión 2.0 de la placa), se recomienda:

- **Sensado en Low-Side:** Colocar la R_{sense} entre la Carga y GND. Esto permite que el operacional trabaje cerca de 0V, eliminando el error de modo común.
- **Uso de un Monitor de Corriente Dedicado:** Integrados como el **INA169** o **INA219** están diseñados específicamente para medir en el lado positivo sin estos errores.

Recepción y envío de datos por serdev

Previo a este paso se verifico el funcionamiento del código de la Pico usando una pagina web comunicando la Raspberry Pi 4 con la Pico por UART sin uso del serdev. El código de la Pico es funcional en su parte UART tanto para la recepción como para el envío de información.

Como la configuración del driver se carga en memoria RAM al apagar el sistema esta se elimina, para evitar el uso constante de ingreso de comandos se uso un scrip de **.sh** llamado **iniciar_scrip.sh** este archivo se ejecuta de la siguiente forma:

./iniciar_scrip.sh

Si todo esta bien se vera lo siguiente:

```
wilson@alan:~/EGB/serdev $ ./iniciar_scrip.sh
--- Iniciando configuración de EGB UART ---
[OK] Módulo cargado correctamente.
[OK] Overlay aplicado correctamente.
mknod: /dev/egb: File exists
[OK] Dispositivo /dev/egb creado con Major: 236
--- Sistema listo para usar con Python o Web ---
```

Luego podrá usarse un scrip en Python, la consola o una pagina web. El sato de seteo se envía en formato “S23.5d” y el dato de lectura tiene el formato “V:XX.X,I:YY.Y”. Todo el contenido para telemetria esta dentro de esta sub carpeta llamada **serdev**.

Telemetria de datos por consola

En esta telemetria usamos el driver serdev que creamos y su uso se basara desde la consola.

Comando	Utilidad
<i>cat /dev/egb</i>	Muestra en pantalla lo recibido desde la pico
<i>echo "S12.7" sudo tee /dev/egb</i>	Envía el dato de seteo a la pico

Este comando filtra la basura dejando solo la tensión y corriente de salida, mostrando los valores cada 1 segundo, para salir solo basta con pulsar ctrl+c. Este comando es mas preciso que el anterior.

```
cat /dev/egb | stdbuf -oL tr -cd '[:print:]\n' | stdbuf -oL grep -E "V:|I:" | while read -r line; do
echo "$(date +%H:%M:%S) -> $line"; sleep 1; done
```

Envío del seteo por consola:

```
wilson@alan:~/EGB/serdev $ echo "S15.6" | sudo tee /dev/egb
S15.6
```

Telemetria de datos por scrrip de Python

Este apartado utiliza el serdev pero ahora con un script que organiza la telemetria de manera mas eficiente. El sccript se llama *monitor_pico.py* y esta dentro de la misma sub carpeta que el que el driver, la sub carpeta se llama *serdev*. Para ejecutar el scrip solo basta con el comando:

python3 monitor_pico.py

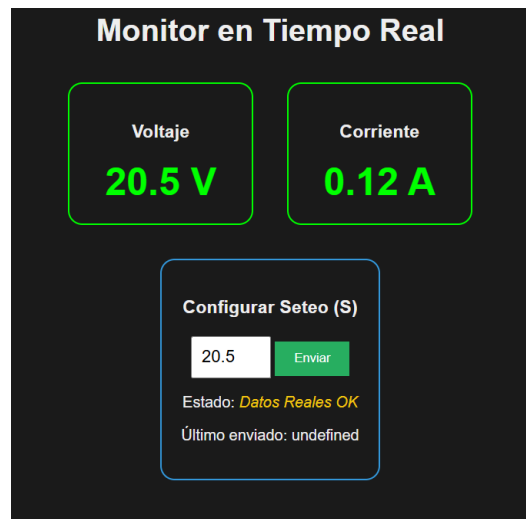
```
wilson@alan:~/EGB/serdev $ python3 monitor_pico.py
=====
CONTROL DE FUENTE RPI -> PICO
=====
Instrucciones:
- Ingresa el voltaje (ej: 15.4)
- Escribe 'mon' para ver datos 10 segundos
- Ctrl+C para salir

Comando o Voltaje > 20.0
Sent: S20.0 ... Esperando respuesta
Confirmación -> V: 12.00V
Confirmación -> V: 11.99V
Confirmación -> V: 11.99V
Comando o Voltaje > mon
--- Monitoreando 10s ---
-> 11:41:55 | V: 12.01V | I: 0.08A
-> 11:41:56 | V: 11.99V | I: 0.08A
-> 11:41:57 | V: 12.00V | I: 0.08A
-> 11:41:58 | V: 12.05V | I: 0.08A
-> 11:41:59 | V: 12.02V | I: 0.08A
-> 11:42:00 | V: 11.99V | I: 0.08A
-> 11:42:01 | V: 12.01V | I: 0.08A
-> 11:42:02 | V: 12.00V | I: 0.08A
-> 11:42:03 | V: 12.00V | I: 0.08A
-> 11:42:04 | V: 12.29V | I: 0.08A
--- Fin monitoreo ---
```

Telemetria de datos por web

La pagina web permite enviar el seteo y recibir la medición de tensión y corriente de salida. La pagina web se almaceno dentro de la sub carpeta *serdev*, el archivo a ejecutar es *app1.py*. El comando para poenr en funcionamiento la pagina web es:

python3 app1.py



NOTA: Configurar el seteo de 12V para arriba no se puso limitadores en la pagina web.